

# Glossary of Key Terms

*Machine Learning for Beginners*

AI Foundations Series — Book 1

---

This glossary collects the main terms and concepts introduced in Book 1, arranged alphabetically for quick reference. Each definition reflects how the term is used in the book itself — plain language first, with just enough precision to be useful when reviewing a chapter or returning to a companion script. Beginners can read it straight through; reviewers can jump to any entry.

## A

---

### Activation Function

A small rule applied to a neuron's output that lets a network learn non-linear patterns. Book 1 uses four: ReLU, sigmoid, tanh, and softmax — each with a natural home in a network.

## B

---

### Backward Propagation (Backpropagation)

The process of learning from mistakes: after a prediction, the network measures its error and works backward through the layers to compute how each weight contributed, producing the gradients used to improve it.

### Batch

A group of training examples processed together in one pass before the weights are updated. See also Mini-Batch.

### Bias

A learnable number added inside a neuron alongside the weighted inputs. Like the weights, it starts as a near-random guess and is nudged during training until the neuron's output becomes useful.

## C

---

### Classification

A task where the network chooses a category (for example, which digit an image shows) rather than predicting a continuous number. Contrast with Regression.

### Cross-Entropy Loss

The standard loss function for classification. In PyTorch, `nn.CrossEntropyLoss` applies the softmax conversion internally, so the network's final layer outputs raw scores rather than probabilities.

## D

---

### Deep Network

A neural network with several hidden layers stacked between input and output. Depth lets simple features detected early combine into more complex features later.

### Dropout

A regularization technique that randomly switches off a fraction of neurons during training, discouraging the network from relying too heavily on any one path and helping it generalize.

## E

---

### Early Stopping

Halting training once performance on the validation set stops improving, preventing the network from continuing to memorize the training data.

### Epoch

One complete pass through the entire training dataset. Watching the loss fall from one epoch to the next is the clearest sign that a network is learning.

## F

---

### Feature

A single measurable input variable describing an example (such as the size or location of a house). Multiple features together form the input the network reads.

### Feature Scaling

Adjusting input features so they share a similar range, so that no single large-valued feature dominates the others during learning. See Normalization and Standardization.

### Flattening

Reshaping a multi-dimensional input — such as a 2-D grid of image pixels — into a single long row of numbers so it can enter a standard fully-connected layer. Convenient, but it discards spatial structure.

### Forward Propagation

Making a prediction: input data flows forward through each layer, getting transformed by weights, biases, and activations until the network produces an output.

## G

---

### Generalization

A model's ability to perform well on new, unseen data rather than just the examples it was trained on. The central goal that dropout, weight decay, and early stopping all serve.

### Gradient

A number indicating the direction and degree to which changing a particular weight would reduce the loss — in short, the direction of improvement for that weight.

### Gradient Descent

The core learning method: weights are repeatedly adjusted a small step in the downhill direction indicated by the gradients, gradually reducing the loss.

## H

---

### Hidden Layer

A layer of neurons between the input and output. Even one hidden layer lets a network learn patterns a single neuron cannot.

### Hyperparameter

A setting chosen by the practitioner before training — such as the learning rate, number of layers, or batch size — rather than learned by the network itself.

## L

---

### Learning Rate

A small factor that scales how big a step the optimizer takes when updating weights. Too large and training is unstable; too small and it crawls.

### Logit

A raw, unnormalized score produced by the final layer before softmax turns it into a probability.

### Loss Function

A measure of how wrong the network's predictions are. Training works to make this number as small as possible; a low loss earned only by memorizing, however, signals overfitting.

## M

---

### Mini-Batch

A small group of training examples used to make one weight update. Splitting data into mini-batches turns a single update per pass into many, speeding up and stabilizing learning.

### MNIST

A standard dataset of seventy thousand images of handwritten digits, used as Book 1's capstone project for recognizing digits.

## N

---

### Neuron

The basic computing unit of a network. It multiplies its inputs by weights, adds a bias, and applies an activation function to produce an output.

### Normalization

Bringing every input feature onto a similar, modest range before it enters the network — described in the book as one of the single most reliable ways to keep training stable.

## O

---

### One-Hot Encoding

Representing a category as a vector of zeros with a single one marking the correct class, so categorical labels can be used numerically.

### Optimizer

The component that actually updates the weights, subtracting a fraction of each gradient (scaled by the learning rate) so the network improves after every step.

### Overfitting

When a model memorizes its training data — scoring well there but poorly on new data. Knowing whether a model overfits is the first step toward fixing it.

## P

---

## Parameters

The learnable values inside a network — its weights and biases. Deeper networks have more parameters and generally need more data to train well.

## PyTorch

The deep-learning library used throughout Book 1 to define networks, run forward and backward passes, and update weights.

# R

---

## ReLU (Rectified Linear Unit)

The book's most-used activation: it outputs zero for negative inputs and passes positive inputs through unchanged. Simple and effective in hidden layers.

## Regression

A task where the network predicts a continuous number (such as a house price) rather than choosing a category. Contrast with Classification.

## Regularization

Any technique that discourages a model from overfitting, including dropout, weight decay, and early stopping.

# S

---

## Sigmoid

An S-shaped activation that squashes any input into the range between 0 and 1, often used to express a probability-like output.

## Softmax

An activation that turns a set of raw scores (logits) into probabilities that sum to one — the standard final step for multi-class classification.

## Standardization

Rescaling a feature so it has roughly zero mean and unit spread, a common form of feature scaling that helps training proceed smoothly.

# T

---

## Tanh

An S-shaped activation similar to sigmoid but squashing inputs into the range between -1 and 1.

## Tensor

The container that holds data flowing through a network — from a single number to a whole batch of examples. Picturing a tensor is key to picturing how data moves through every program in the book.

## Test Set

Data held back and used only once, at the very end, to judge final performance honestly. Part of the three-way split alongside training and validation sets.

## Training Curve

A plot of the loss (and often accuracy) over the course of training, read to tell whether a network is learning, has converged, or is overfitting.

## Training Set

The portion of data the network actually learns from by adjusting its weights.

## V

---

### Validation Set

Data set aside to check performance during training without being learned from — used to tune choices and decide when to stop.

### Vanishing-Gradient Problem

A difficulty in deeper networks where gradients shrink toward zero as they propagate backward, slowing or stalling learning in the earliest layers. Part of why ReLU is preferred over saturating activations.

## W

---

### Weight

A learnable multiplier applied to an input inside a neuron. Weights begin as essentially random guesses and are nudged a little at a time until the network's outputs are useful.

### Weight Decay

A regularization technique that gently penalizes large weights, nudging the network toward simpler solutions that generalize better.

### Weight Initialization

The choice of starting values for a network's weights. Sensible initial scaling matters because poor starting weights can make training unstable from the outset.

---

© 2026 David Grunwald. All rights reserved.